Group Homework: Implementing Text Generation Strategies

CSE 368 – Artificial Intelligence

Group size: 3 students **Total:** 100 points
Submission date: Tuesday 03 November 2026 **Submission:** `.py` or `.ipynb`

In this assignment, you will implement and compare three strategies used by language models to select the next token during text generation:

1. Greedy decoding
2. Random sampling
3. Temperature-based sampling

You will work in groups of three to investigate how different decoding strategies affect the tokens selected by a language model. You do not need to train a neural network or build a Transformer from scratch. Instead, you will use predefined model logits to simulate the next-token selection stage of a language model.

1 Background

A language model assigns a score, called a *logit*, to each possible next token. Logits are converted into probabilities using the softmax function. For logits $z_1, z_2, \ldots, z_n$, the probability of selecting token i is

$$P(i) = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}.$$

The resulting probabilities lie between 0 and 1 and sum to 1.

1.1 Greedy Decoding

Greedy decoding selects the token with the highest logit, which is also the token with the highest softmax probability:

$$i^* = \arg\max_i z_i.$$

When the logits remain unchanged, greedy decoding always selects the same token.

1.2 Random Sampling

Random sampling selects a token according to the probability distribution produced by softmax. A token with a higher probability is more likely to be selected, but tokens with lower probabilities may also be selected.

Random sampling is not the same as uniformly selecting a random token. Selection probabilities must be based on the model's logits.

1.3 Temperature-Based Sampling

Temperature-based sampling adjusts the logits before applying softmax. The temperature parameter T controls how concentrated or spread out the probability distribution becomes:

$$P(i \mid T) = \frac{e^{z_i/T}}{\sum_{j=1}^{n} e^{z_j/T}}.$$

The effects of temperature are:

- $T < 1$**:** The distribution generally becomes more concentrated around the highest-scoring tokens.
- $T = 1$**:** The distribution is the same as ordinary softmax sampling.
- $T > 1$**:** The distribution becomes flatter, making lower-scoring tokens relatively more likely.

Temperature must be greater than zero.

2 Provided Data

For this assignment, assume that a language model produces the following tokens and logits:

Token	**Logit**
cat	2.5
dog	1.8
robot	0.7
student	0.2
car	−0.5

These logits are simulated model outputs, not probabilities. Your program must convert them into probabilities using softmax. Use the same tokens and logits when comparing all three decoding strategies.

3 Programming Tasks

3.1 Task 1: Implement the Softmax Function (15 points)

Complete the `softmax()` function in your Jupyter Notebook. Your function must:

- Accept an array of logits.
- Convert the logits into probabilities.
- Return probabilities that sum to approximately 1.
- Use a numerically stable implementation by subtracting the maximum logit before calculating exponentials.

Print the resulting probability for each token.

3.2 Task 2: Implement Greedy Decoding (10 points)

Complete the `greedy_decode()` function in your Jupyter Notebook. Your function must accept the token list and logits, identify the token with the highest logit, and return the selected token. Run the function several times and verify that the same token is selected when the logits remain unchanged.

3.3 Task 3: Implement Random Sampling (15 points)

Complete the `random_sampling()` function in your Jupyter Notebook. Your function must:

- Calculate probabilities using softmax.
- Select one token randomly according to those probabilities.
- Return the selected token.

Run the function multiple times and observe whether different tokens are selected. Do not select tokens uniformly at random; use the calculated probability distribution.

3.4 Task 4: Implement Temperature-Based Sampling (20 points)

Complete the `temperature_sampling()` function in your Jupyter Notebook. Your function must:

- Accept the token list, logits, and a temperature value.
- Divide each logit by the temperature.
- Apply softmax to the scaled logits.
- Sample one token according to the resulting probabilities.
- Return the selected token.
- Reject temperatures less than or equal to zero by raising an appropriate error.

Test the function with temperatures $T = 0.5$, $T = 1.0$, and $T = 2.0$. Print the probability distribution for each temperature and explain how the distribution changes.

3.5 Task 5: Conduct Experiments and Compare Results (15 points)

Run the following experiments using the provided tokens and logits.

Experiment A: **Greedy decoding:** Run greedy decoding 1,000 times and count how many times each token is selected.

Experiment B: **Random sampling:** Run random sampling 1,000 times and record the number and percentage of selections for each token.

Experiment C: **Temperature-based sampling:** Run temperature-based sampling 1,000 times for each temperature: $T = 0.5$, $T = 1.0$, and $T = 2.0$. Record the number and percentage of selections for each token.

Use a fixed random seed, such as `np.random.seed(42)`, to make your experiments reproducible. Present your results in a table similar to the following:

Token	**Greedy** (%)	**Random** (%)	$T = 0.5$ (%)	$T = 1.0$ (%)	$T = 2.0$ (%)
cat					
dog					
robot					
student					
car					

The percentages for each method should add up to approximately 100%, allowing for rounding. Create a bar chart or another appropriate visualization comparing the observed token-selection frequencies across the temperature settings.

3.6 Task 6: Analyze Your Results (3*5 = 15 points)

Answer the following questions in Markdown cells in your Jupyter Notebook:

1. Which token is selected by greedy decoding, and why?
2. Why is random sampling different from uniformly selecting a random token?
3. How does lowering the temperature affect the probability distribution?
4. Why might a language model use sampling instead of greedy decoding?
5. What are two advantages and two disadvantages of using a higher temperature for text generation?

Support your answers with observations from your experiments.

4 Group Organization and Responsibilities

Each group must contain three students. Use the following initial division of responsibilities:

- **Student 1:** Implement and test the softmax function and greedy decoding.
- **Student 2:** Implement and test random sampling and temperature-based sampling.
- **Student 3:** Conduct the experiments, prepare the comparison table or chart, and coordinate the written report.

All group members are responsible for understanding the complete program and the concepts covered in the assignment. Include a brief Markdown cell in the notebook describing each member's contribution.

5 Submission Requirements

Submit **one completed Jupyter Notebook file (`.ipynb`) per group**. Do not submit a separate Python file, PDF, Word document, or separate image/report files.

Your notebook must contain all of the following:

1. **Python implementation:** Completed functions for softmax, greedy decoding, random sampling, and temperature-based sampling.

2. **Experiment results:** Executed code showing the probability distributions and token-selection frequencies for all required methods and temperatures.
3. **Results table:** A table comparing token-selection percentages across the required methods.
4. **Visualization:** A bar chart or another appropriate graph generated and displayed inside the notebook.
5. **Written analysis:** Answers to all six analysis questions, entered in Markdown cells.
6. **Group contribution statement:** A Markdown cell listing all three group members and briefly describing each member's contribution.

Make sure the notebook runs from beginning to end without errors. Include comments in your code and explanatory Markdown cells so that your work can be understood without a separate report. Before submitting, restart the notebook kernel and run all cells to verify that the outputs are present and reproducible.

6 Grading Rubric

Component	Points
Softmax implementation	15
Greedy decoding implementation	10
Random sampling implementation	15
Temperature-based sampling and input validation	20
Experiments and presentation of results	15
Written analysis and explanations	15
Code readability and group contribution statement	10
Total	**100**

7 Important Notes

- Use Python and NumPy for the implementation.
- You may consult course materials and official Python or NumPy documentation.
- Do not train a neural network or download a pretrained language model for the required tasks.
- Your experiments must use the provided tokens and logits.
- Your results should be reproducible using a fixed random seed.
- Explain your own experimental observations rather than submitting code alone.

The purpose of this assignment is to understand how different decoding strategies affect next-token selection. This assignment simulates the token-selection stage of language generation; it does not implement or train the Transformer itself.